

Blob store is a storage solution for unstructured data. We can store photos, audio, videos, binary executable codes, or other multimedia items in a blob store. Every type of data is stored as a **blob**. It follows a flat data organization pattern where there are no hierarchies, that is, directories, sub-directories, and so on.

Mostly, it's used by applications with a particular business requirement called **write once, read many (WORM)**, which states that data can only be written once and that no one can change it. Just like in Microsoft Azure, the blobs are created once and read many times. Additionally, these blobs can't be deleted until a specified interval, and they also can't be modified to protect critical data.

Why do we use a blob store?

Blob store is an important component of many data-intensive applications, such as YouTube, Netflix, Facebook, and so on. The table below displays the blob storage used by some of the most well-known applications. These applications generate a huge amount of unstructured data every day. They require a storage solution that is easily scalable, reliable, and highly available, so that they can store large media files. Since the amount of data continuously increases, these applications need to store an unlimited number of blobs. According to some estimates, YouTube requires more than a petabyte of additional storage per day. In a system like YouTube, a video is stored in multiple resolutions. Additionally, the video in all resolutions is replicated many times across different data centers and regions for availability purposes. That's why the total storage required per video is not equal to the size of the uploaded video.

Functional requirements

Here are the functional requirements of the design of a blob store:

Create a container: The users should be able to create containers in order to group blobs. For example, if an application wants to store user-specific data, it should be able to store blobs for different user accounts in different containers. Additionally, a user may want to group video blobs and separate them from a group of image blobs. A single blob store user can create many containers, and each container can have many blobs, as shown in the following illustration. For the sake of simplicity, we assume that we can't create a container inside a container.

Put data: The blob store should allow users to upload blobs to the created containers.

Get data: The system should generate a URL for the uploaded blob, so that the user can access that blob later through this URL.

Delete data: The users should be able to delete a blob. If the user wants to keep the data for a specified period of time (retention time), our system should support this functionality.

List blobs: The user should be able to get a list of blobs inside a specific container.

Delete a container: The users should be able to delete a container and all the blobs inside it.

List containers: The system should allow the users to list all the containers under a specific account.

Non-functional requirements

Here are the non-functional requirements of a blob store system:

Availability: Our system should be highly available.

Durability: The data, once uploaded, shouldn't be lost unless users explicitly delete that data.

Scalability: The system should be capable of handling billions of blobs.

Throughput: For transferring gigabytes of data, we should ensure a high data throughput.

Reliability: Since failures are a norm in distributed systems, our design should detect and recover from failures promptly.

Consistency: The system should be strongly consistent. Different users should see the same view of a blob.

★ Estimation

- 5m users
- Rps is 500 per server
- avg video size = 50 MB
- avg thumbnail = 20 KB
- videos per day = 250,000
- read req per user per day = 20

$$\begin{aligned}\text{Servers req} &= \frac{\text{no. of Rps}}{\text{Rps of server}} = \frac{5m}{500} = \frac{5 \times 10^6}{500} \\ &= 10k \text{ servers}\end{aligned}$$

★ Storage

$$\begin{aligned}\text{Total storage/day} &= \text{No. of videos/day} \times \\ &\quad \text{Storage/video} + \text{Storage/thumbnail} \\ &= 12.51 \text{ TB/day}\end{aligned}$$

★ Bandwidth

$$\begin{aligned}\text{Total bw} &= \frac{\text{Total size per day}}{24 \times 60 \times 60} \\ &= \frac{12.51 \text{ TB}}{86400} \\ &= 1.16\end{aligned}$$

- outgoing traffic

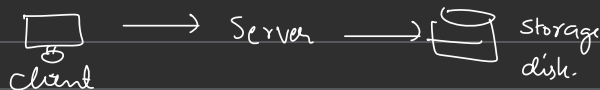
$$\begin{aligned} \text{Total bw} &= \frac{\text{No of active users} \times \text{No of RPS}}{\text{seconds in a day}} \\ &= \frac{5m \times 20}{50} \\ &= 462.96 \end{aligned}$$

$$\text{Total bw} = 462.96 \text{ Gbps}$$

Building blocks

- Rate Limiter
- LB
- DB
- monitoring

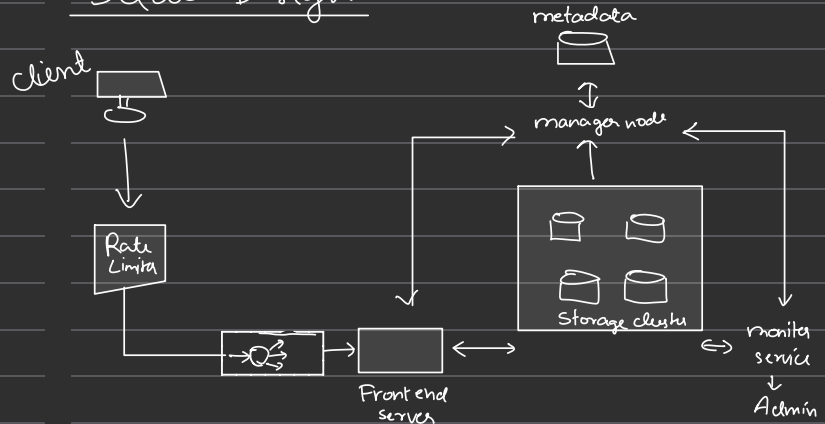
High Level



* API disk:

- ContainerDisk
- putBlob (container, path, blobname, data)
- getBlob (blobpath)
- deleteBlob (path)
- listBlob (container, path)
- deleteContainer (container, path)

Detail Design



Client: This is a user or program that performs any of the API functions that are specified.

Rate limiter: A rate limiter limits the number of requests based on the user's subscription or limits the number of requests from the same IP address at the same time. It doesn't allow users to exceed the predefined limit.

Load balancer: A load balancer distributes incoming network traffic among a group of servers. It's also used to reroute requests to different regions depending on the location of the user, different data centers within the same region, or different servers within the same data center. **DNS** load balancing can be used to reroute the requests among different regions based on the location of the user.

Front-end servers: Front-end servers forward the users' requests for adding or deleting data to the appropriate storage servers.

Data nodes: Data nodes hold the actual blob data. It's also possible that they contain a part of the blob's data. Blobs are split into small, fixed-size pieces called **chunks**. A data node can accommodate all of the chunks of a blob or at least some of them.

Manager node: A manager node is the core component that manages all data nodes. It stores information about storage paths and the access privileges of blobs. There are two types of access privileges: private and public. A *private* access privilege means that the blob is only accessible by the account containing that blob. A *public* access privilege means that anyone can access that blob.

Metadata storage: Metadata storage is a distributed database that's used by the manager node to store all the metadata. Metadata consists of account metadata, container metadata, and blob metadata.

Account metadata contains the account information for each user and the containers held by each account.

Container metadata consists of the list of the blobs in each container.

Blob metadata consists of where each blob is stored. The blob metadata is discussed in detail in the next lesson.

Monitoring service: A monitoring service monitors the data nodes and the manager node. It alerts the administrator in case of disk failures that require human intervention. It also gets information about the total available space left on the disks to alert administrators to add more disks.

Administrator: An administrator is responsible for handling notifications from the monitoring services and conducting routine checkups of the overall service to ensure reliability.

The architecture of how these components interconnect is shown in the diagram below:

Reading a blob

When a read request for a blob reaches the front-end server, it asks the manager node for that blob’s metadata.

The manager node first checks whether that blob is private or public, based on the path of the blob and whether we’re authorized to transfer that blob or not.

After authorizing the blob, the manager node looks for the chunks for that blob in the metadata and looks at their mappings to data nodes. The manager node returns the chunks and their mappings (data nodes) to the client.

The client then reads the chunk data from the data nodes.

Before we answer the questions listed above, let’s look at how we create layers of abstractions for the user to hide the internal complexity of a blob store. These abstraction layers help us make design-related decisions as well.

There are three layers of abstractions:

User account: Users uniquely get identified on this layer through their `account_ID`. Blobs uploaded by users are maintained in their containers.

Container: Each user has a set of containers that are all uniquely identified by a `container_ID`. These containers contain blobs.

Blob: This layer contains information about blobs that are uniquely identified by their `blob_ID`. This layer maintains information about the metadata of blobs that’s vital for achieving the availability and reliability of the system.

We can take routing, storage, and sharding decisions on the basis of these layers. The table below summarizes these layers.

Blob metadata

When a user uploads a blob, it’s split into small-sized chunks in order to be able to support the storage of large files that can’t fit in one contiguous location, in one data node, or in one block of a disk associated with that data node. The chunks for a single blob are then stored on different data nodes that have enough storage space available to store these chunks. There are billions of blobs that are kept in storage. The manager node has to store all the information about the blob’s chunks and where they are stored, so that it can retrieve the chunks on reads. The manager node assigns an ID to each chunk.

The information about a blob consists of chunk IDs and the name of the assigned data node for each chunk. We split the blobs into equal-sized chunks. Chunks are replicated to enable them to deal with data node failure. Hence, we also store the replica IDs for each chunk. We have access to all this information pertaining to each blob.

Let’s say we have a blob of 128 MB, and we split it into two chunks of 64 MB each. The metadata for this blob is shown in the following table:

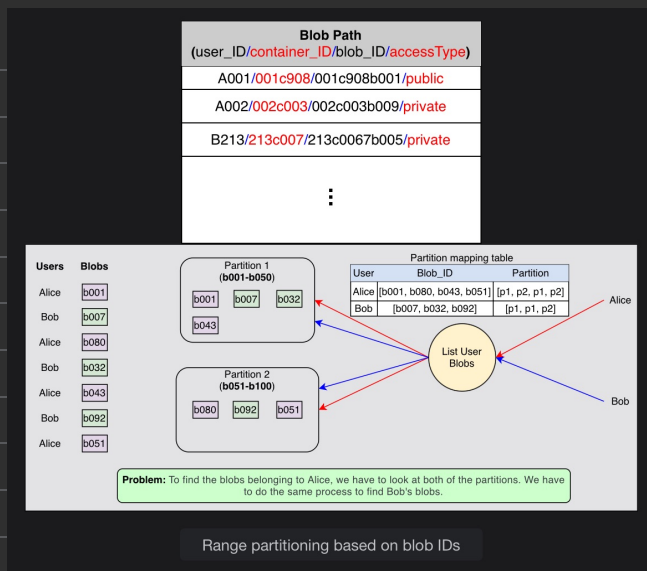
Introduction

Even though we discussed the design of the blob store system and its major components in detail in the previous lesson, a number of interesting questions still require answers. For example, how do we store large blobs? Do we store them in the same disk, in the same machine, or do we divide those blobs into chunks? How many replicas of a blob should be made to ensure reliability and availability? How do we search for and retrieve blobs quickly? These are just some of the questions that might come up.

This lesson addresses these important design concerns. The table below summarizes the goals of this lesson.

Partition data

We talked about the different levels of abstraction in a blob store—the account layer, the container layer, and the blob layer. There are billions of blobs that are stored and read. There is a large number of data nodes on which we store these blobs. If we look for the data nodes that contain specific blobs out of all of the data nodes, it would be a very slow process. Instead, we can group data nodes and call each group a **partition**. We maintain a partition map table that contains a list of all the blobs in each partition. If we distribute the blobs on different partitions independent of their container IDs and account IDs, we encounter a problem, as shown in the following illustration:



Partitioning based on the blob IDs causes certain problems. For example, the blobs under a specific container or account may reside in different partitions that add overhead while reading or listing the blobs linked to a particular account or a particular container.

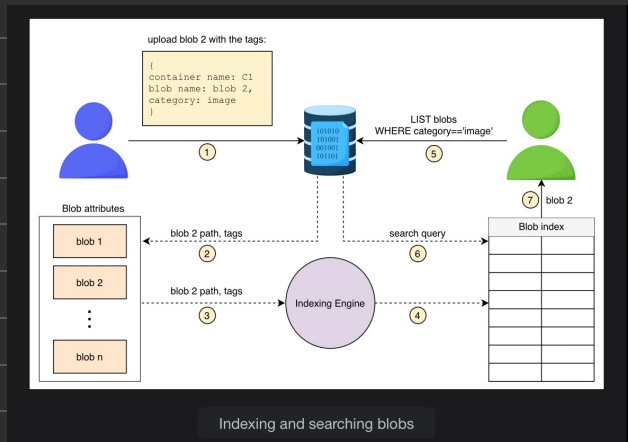
To overcome the problem described above, we can partition the blobs based on the complete path of the blob. The partition key here is the combination of the account ID, container ID, and blob ID. This helps in co-locating the blobs for a single user on the same partition server, which enhances performance.

Blob indexing

Finding specific blobs in a sea of blobs becomes more difficult and time-consuming with an increase in the number of blobs that are uploaded to the storage. The **blob index** solves the problem of blob management and querying.

To populate the blob index, we define key-value tag attributes on the blobs while uploading the blobs. We use multiple tags, like container name, blob name, upload date and time, and some other categories like the image or video blob, and so on.

As shown in the following illustration, a blob indexing engine reads the new tags, indexes them, and exposes them to a searchable blob index:

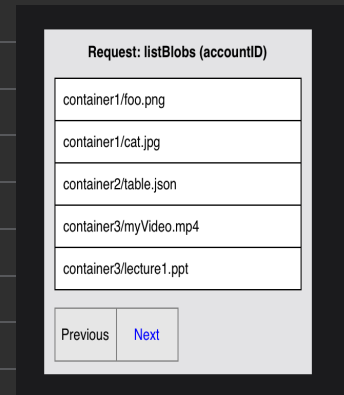


Pagination for listing

Listing is about returning a list of blobs to the user, depending on the user's entered prefix. A prefix is a character or string that returns the blobs whose name begins with that particular character or string.

Users may want to list all the blobs associated with a specific account, all the blobs present inside a specific container, or they may want to list some public blobs based on a prefix. The problem is that this list could be very long. We can't return the whole list to the user in one go. So, we have to return the list of the blobs in parts.

Let's say a user wants a list of blobs associated with their account and there are a total of 2,000 blobs associated with that account. Searching, returning, and loading too many blobs at once affects performance. This is where paging becomes important. We can return the first five results and give users a next button. On each click of the next button, it returns the next five results. This is called pagination.



Replication

Replication is carried out on two levels to support availability and strong consistency. To keep the data strongly consistent, we synchronously replicate data among the nodes that are used to serve the read requests, right after performing the write operation. To achieve availability, we can replicate data to different regions or data centers after performing the write operation. We don't serve the read request from the other data center or regions until we have not replicated data there.

These are the two levels of replication:

Synchronous replication within a storage cluster.

Asynchronous replication across data centers and regions.

Synchronous replication within a storage cluster

A **storage cluster** is made up of N racks of storage nodes, each of which is configured as a fault domain with redundant networking and power.

We ensure that every data written into a storage cluster is kept durable within that storage cluster. The manager node maintains enough data replicas across the nodes in distinct fault domains to ensure data durability inside the cluster in the event of a disk, node, or rack failure.

Success can be returned to the client once a write has been synchronously replicated inside that storage cluster. This allows for **quick writes** because:

We replicate data within a storage cluster where all the nodes are nearby, thus reducing the latency.

We use in-lined data copying where we use redundant network paths to copy data to all the replicas in parallel.

This replication technique helps maintain data consistency and availability inside the storage cluster.

Asynchronous replication across data centers and region

The blob store's data centers are present in different regions—for example, Asia-Pacific, Europe, eastern US, and more. In each region, we have more than one data center placed at different locations, so that if one data center goes down, we have other data centers in the same region to step in and serve the user requests. There are a minimum of three data centers within each region, each separated by miles to protect against local events like fires, floods, and so on.

The number of copies of a blob is called the **replication factor**. Most of the time, a replication factor of **three** is sufficient.

We keep three copies of a blob. One is the local copy within the data center in the primary region to protect against server rack and drive failures. The second copy of the blob is placed in the other data center within the same region to protect against fire or flooding in the data center. The third copy is placed in the data center of a different region to protect against regional disasters.

Garbage collection while deleting a blob

Since the blob chunks are placed at different data nodes, deleting from many different nodes takes time, and holding a client until that's done is not a viable option. Due to real-time latency optimization, we don't actually remove the blob from the blob store against a delete request. Instead, we just mark a blob as "DELETED" in the metadata to make it inaccessible to the user. The blob is removed later after responding to the user's delete request.

Marking the blob as deleted, but not actually deleting it at the moment, causes internal metadata inconsistencies, meaning that things continue to take up storage space that should be free. These metadata inconsistencies have no impact on the user. For example, for a blob marked as deleted in the metadata, we still have the entries for that blob's chunks in the metadata. The data nodes are still holding on to that blob's chunks. Therefore, we have a service called a **garbage collector** that cleans up metadata inconsistencies later. The deletion of a blob causes the chunks associated with that blob to be freed. However, there could be an appreciable time delay between the time a blob is deleted by a user and the time of the corresponding increase in free space in the blob store. We can bear this appreciable time delay because, in return, we have a real-time fast response benefit for the user's delete blob request.

The whole deletion process is shown in the following illustration

Cache the blob store

Caching can take place at multiple levels. Below are some examples:

The metadata for a blob's chunks is cached on the client side when it's read for the first time. The client can go directly to the data nodes without communicating to the manager node to read the same blob a second time.

At the **front-end servers**, we cache the partition map and use it to determine which partition server to forward each request to.

The frequently accessed chunks are cached at the **manager node**, which helps us stream large objects efficiently. It also reduces disk I/O.

Availability

The replication part of our design makes the system available. For reading the data, we keep four replicas for each blob. Having replicas, we can distribute the request load. If one node fails, the other replica node can serve the request. Moreover, our replica placement strategy handles a whole data center failure and can even handle the situation of region unavailability due to natural disasters. We ensure that enough replicas are available at any point in time using our monitoring service, which makes a copy of the data in a timely manner if the number of failed replicas exceeds the specified threshold.

For write requests, we replicate the data within the cluster in a fault-tolerant way and quickly respond to the user, making the system available for write requests.

To keep the manager node available, we keep a backup of its state. In the case of a manager-node failure, we start a new instance of the manager node, initializing it from the saved state.

Durability

The replication and monitoring services ensure the durability of the data. The data, once uploaded, is synchronously replicated within a storage cluster. If data loss occurs at one node, we can recover the data from the other nodes. The monitoring service monitors the storage disks. If any disk fails, the monitoring service alerts the administrators to change the disk and sends messages to the manager node to copy the content on that disk on to the other available disk or the newly added disk. The manager node then updates the mapping accordingly.

Scalability

The partitioning and splitting of blobs into small-sized chunks helps us scale for billions of blob requests. Blobs are partitioned into separate ranges and served by different partition servers. The partition mappings specify which partition server will serve which particular blob range requests. Partitioning also provides automatic load balancing between partition servers to fulfill the blobs' traffic needs.

Our system is horizontally scalable for storage. As need for storage arises, we add more data nodes. At some point, though, our manager node can become a bottleneck. A single manager node can handle 10,000 queries per second (QPS).

Throughput

We save chunks of a blob on different data nodes that distribute the requests for a blob to multiple machines. Parallel fetching of chunks from multiple data nodes helps us achieve high throughput.

Additionally, caching at different layers— the client side, front-end servers, and the manager node—improves our throughput and reduces latency.

Reliability

We achieve reliability through our monitoring techniques. For example, the heartbeat protocol keeps the manager node updated on the status of data nodes. This enables the manager node to request data from reliable nodes only. Furthermore, it takes necessary precautions to ensure reliable service. For example, the failure of a node triggers the manager node to request an additional replica node.

The monitoring services also alert the administrators to change faulty hardware like failed disks or broken network links or switches. To ensure a reliable service, the manager node keeps track of the available space on the disk. If the available space left reaches a certain threshold, an alert is sent to the administrators to add new disks.

Consistency

We synchronously replicate the disk data blocks inside a storage cluster upon a write request, making the data strongly consistent inside the storage cluster. This is done on the user's critical path. We serve the subsequent read requests on this data from the same storage cluster until we have replicated this data in another data center and or on other storage clusters.

After responding to the write request and replicating data within the storage cluster, we asynchronously replicate blobs in the data centers placed far away or in other regions to ensure availability